

 MVP RNDT — Catálogo de Projetos Ágeis e Prompts LLM

Rede Nacional de Dados de Transporte | Data Space de Transporte do Brasil Versão:
2.0 | Modelo: Squads Paralelos (30 Dias) | Junho/2026

 ÍNDICE RÁPIDO

Seção	Para quem	Quando usar
Seção 0: IDD	Arquiteto + Tech Leads	Dia 0-7
Projeto	Squad 1 (Infra)	Dia 1-15
INFRA-CORE-001		
Projeto	Squad 2 (Data)	Dia 1-15
DATA-PROV-002		
Projeto	Squad 3 (Consumer)	Dia 1-15
CONS-UFPE-003		
Projeto	Squad 4 (Gov/Portal)	Dia 1-30
GOV-PORTAL-004		
Projeto	Todos os squads	Dia 15-30
INTEG-GOLIVE-005		
Anexos	Todos	Consulta contínua

 SEÇÃO 0: INTERFACE DEFINITION DOCUMENT (IDD)

“Sem contratos de interface, não há paralelismo. Sem paralelismo, não há 30 dias.”

0.1 O que é o IDD?

O **Interface Definition Document** é o contrato técnico que permite que 4 squads trabalhem em paralelo sem depender uns dos outros. Ele define **formatos, endpoints, schemas e mocks** antes que qualquer código seja escrito.

0.2 Cronograma do IDD

Dia	Atividade	Responsável	Saída
0	Workshop de Alinhamento (4h)	Arquiteto RNDT + 4 Tech Leads	IDD v0.1 (esboço)
1-2	Refinamento por Interface	Cada squad revisa o que consome/ produz	IDD v1.0 (rascunho)
3-5	Implementação de Mocks	Squad 1 (infra) + Squad 4 (gov)	Mock Server operacional
6	Teste de Integração contra Mocks	Todos os squads	Relatório de compatibilidade
7	GATE DE VALIDAÇÃO	Arquiteto RNDT + POs	IDD v1.2 (baseline)

Regra de ouro: Nenhum squad inicia desenvolvimento de código antes do Gate do Dia 7.

0.3 Checklist do Gate (Dia 7)

- Squad 2 obtém token JWT do mock Keycloak e valida claims (sub, rndtStatus, securityProfile)
- Squad 3 consome catálogo DSP do mock e parseia JSON-LD (@context, dcat:dataset)
- Squad 4 renderiza tema CKAN com metadados do mock (dct:title, dcat:accessService)
- Squad 1 faz deploy do Helm Chart base em cluster de teste
- Todos os squads trocam mensagens DSP (request/response) via mock
- Postman Collection de referência executa com 100% de sucesso
- Documentação de troubleshooting do mock está disponível

Se falhar → volta para Dia 1–2. Não há exceção.

0.4 Onde Publicar

```
Repositório: https://github.com/rndt/mvp-idd
├─ idd/v1.2/
│   ├── 01-identidade.md
│   ├── 02-catalogo.md
│   ├── 03-negociacao.md
│   ├── 04-transferencia.md
│   ├── 05-auditoria.md
│   └─ 06-infraestrutura.md
├─ mocks/
│   ├── keycloak-omejdn/
│   ├── dsp-mock-server/
│   └─ postman/
│       └─ rndt-reference-collection.json
└─ CHANGELOG.md
```

 PROJETO INFRA-CORE-001 | Squad 1: Infra & Core

Objetivo

Subir os serviços centrais da RNDT em Kubernetes até o **Dia 15**.

Entregáveis

#	Entregável	Prazo	Critério de Aceite
1.1	Keycloak (DAPS) operacional	D7	Realm RNDT-MVP, mTLS ICP-Brasil, tokens JWT
1.2	CKAN + Solr + PostgreSQL	D10	Portal acessível, harvest configurado
1.3	Clearing House (Fraunhofer)	D12	API REST, validação JWS, Merkle Tree
1.4	Apache Superset	D15	Conectado aos bancos, dashboards base
1.5	mTLS entre componentes	D15	Certificados válidos, conectividade testada

Contratos de Interface (entregues para outros squads)

Componente	URL	Contrato
Keycloak	https://identity.rndt.gov.br	OAuth2 + mTLS, realm RNDT-MVP
CKAN	https://rndt.gov.br/api/3/action/	REST API, JSON, DCAT
Clearing House	https://clearing.rndt.gov.br/api/v1/	REST API, JSON-LD
Superset	https://dashboards.rndt.gov.br	Web UI, SQL queries

Prompts LLM

Prompt 1.1: Helm Chart — Keycloak (DAPS)

Gere um Helm Chart completo para deploy do Keycloak no Kubernetes, configurado como DAPS para o MVP da RNDT.

Requisitos:

- Realm: "RNDT-MVP"
- Autenticação X.509/mTLS com certificados ICP-Brasil
- Clientes pré-cadastrados:
 - * PRF (CNPJ: 00.394.502/0001-44)
 - * DNIT (CNPJ: 04.898.488/0001-77)
 - * ANTT
 - * UFPE
- Mapeamento de atributos: cnpj → sub, orgao, rndtStatus
- Claims JWT: securityProfile, rndtStatus
- Persistência: PostgreSQL (subchart)
- Ingress com TLS 1.3
- Recursos: requests/limits CPU e memória
- Health checks (liveness/readiness)
- ConfigMap para configs não-sensíveis
- Secret para credenciais e certificados

Saída: values.yaml comentado, templates/, arquivos de teste

Prompt 1.2: Helm Chart — CKAN (Metadata Broker)

Gere um Helm Chart para CKAN 2.10+ no Kubernetes, como Metadata Broker da RNDT.

Requisitos:

- CKAN core + PostgreSQL + Redis + Apache Solr (subcharts)
- Plugin ckanext-dcat pré-instalado
- Plugin ckanext-harvest para colheita
- Harvest sources: PRF e DNIT (URLs endpoints DSP)

- Tema customizado via ConfigMap (placeholder RNDT)
- Ingress com TLS
- Persistência de dados e uploads
- Variáveis de ambiente para Keycloak (SSO futuro)
- Health checks e recursos

Saída: init-container que cria admin user e configura harvesters

Prompt 1.3: Helm Chart — IDS Clearing House

Gere um Helm Chart para o IDS Clearing House do Fraunhofer ISST.

Requisitos:

- Imagem Docker Rust oficial
- Persistência PostgreSQL com criptografia de disco
- API REST para recibos de transferência (JSON-LD)
- Validação JWS (JsonWebSignature2020)
- Merkle Tree para imutabilidade
- Health check endpoint
- ConfigMap com parâmetros de rede RNDT
- Secret com chaves de assinatura
- Backup automático (CronJob)

Prompt 1.4: Helm Chart — Apache Superset

Gere um Helm Chart para Apache Superset no Kubernetes, como Dashboard de Governança.

Requisitos:

- Superset + PostgreSQL + Redis (subcharts)
- Datasources pré-configurados:
 - * CKAN (PostgreSQL) — métricas do portal
 - * Keycloak (PostgreSQL) — métricas de autenticação
 - * Clearing House (PostgreSQL) — métricas de auditoria
- Dashboards iniciais:
 - * "Saúde da RNDT" (nós ativos, contratos, transferências)
 - * "Popularidade de Datasets" (buscas, downloads)
 - * "Auditoria" (logs de transferência, hashes)
- Admin user via init-container
- Ingress com TLS
- Recursos e health checks

PROJETO DATA-PROV-002 | Squad 2: Data & Provider

Objetivo

Ingerir dados PRF/DNIT, transformar para DATEX II e expor via EDC Provedor até o **Dia 15**.

Entregáveis

#	Entregável	Prazo	Critério de Aceite
2.1	Pipeline PRF Sinistros → DATEX II	D10	CSV processado, DATEX II válido, staging
2.2	Pipeline DNIT Pavimentação → DATEX II	D10	CSV processado, DATEX II válido, staging
2.3	EDC Provedor (MinT) deployado	D12	Control Plane + Data Plane, mTLS
2.4	2 Assets cadastrados	D14	asset-prf-sinistros-2026, asset-dnit-pavimentacao-2026
2.5	Políticas ODRL	D15	Templates público e comercial validados

Dependências

Precisa de	Interface	Mitigação até D15
INFRA-CORE-001 (Keycloak)	OAuth2 + mTLS	Usar Omejdn mock local
INFRA-CORE-001 (CKAN)	REST API	Usar nginx + JSON estático
GOV-PORTAL-004	Templates ODRL	Squad 4 entrega D7, Squad 2 usa D8

Prompts LLM

Prompt 2.1: Rota Apache Camel — PRF Sinistros → DATEX II

Gere uma rota Apache Camel (YAML/DSL) para transformar CSVs de sinistros da PRF em DATEX II.

FONTE — CSV com colunas:

```
data_inversa, dia_semana, hora, uf, br, km, municipio,
causa_acidente, tipo_acidente, classificacao_acidente,
fase_dia, sentido_via, condicao_meteorologica, tipo_pista,
tracado_via, uso_solo, pessoas, mortos, feridos_leves,
feridos_graves, ilesos, ignorados, feridos, veiculos,
latitude, longitude
```

DESTINO — DATEX II v3:

```
- d2LogicalModel → situation → situationRecord → accident
- roadNumber (BR), startDistanceFromReferent (km)
- accidentType ← causa_acidente
- injuryStatusType ← mortos/feridos
- weatherRelatedRoadConditions ← condicao_meteorologica
- location → pointCoordinates (lat/lon corrigidos via map-matching)
```

REGRAS:

```
- Agendamento: todo dia às 02:00
- Validação: descartar registros sem BR ou KM
- Correção geográfica: chamar OSRM/GraphHopper para snap
- Particionamento: arquivo por UF e mês
- Staging em MinIO/S3
- Notificação para EDC após processamento
```

Saída: rota principal, processor customizado (Java/Kotlin), testes unitários

Prompt 2.2: Rota Apache Camel — DNIT Pavimentação → DATEX II

Gere uma rota Apache Camel (YAML/DSL) para transformar CSVs de pavimentação do DNIT em DATEX II.

FONTE — CSV com colunas:

br_rodovia, km_inicial, km_final, icm_classificacao,
data_levantamento, tipo_pavimento, condicao_superficie,
largura_faixa, numero_faixas, extensao_km,
latitude_inicial, longitude_inicial, latitude_final, longitude_final

DESTINO — DATEX II v3:

- d2LogicalModel → situation → situationRecord → roadSurfaceCondition
- roadNumber, startDistanceFromReferent, endDistanceFromReferent
- roadSurfaceConditionStatus ← icm_classificacao
- roadSurfaceType ← tipo_pavimento
- numberOfLanes, laneWidth
- location → linearExtension (coordenadas inicial/final)

REGRAS:

- Agendamento: semanal (domingos 03:00)
- Validação: km_inicial < km_final
- Correção geográfica: snap ao eixo rodoviário
- Particionamento: por BR e ano
- Staging em MinIO/S3

Saída: rota, processor, testes

Prompt 2.3: Configuração Eclipse EDC — Provedor

Gere a configuração completa do Eclipse Dataspace Connector (EDC) como Provedor.

Requisitos:

- application.properties:
 - * Control Plane: DSP protocol, management API
 - * Data Plane: HTTP proxy, mTLS
 - * Asset Index: PostgreSQL
 - * Contract Negotiation: ODRL policies
 - * Identity: Keycloak/Omejdn DAPS integration
 - * Transfer Process: HTTP push/pull
- Assets JSON:
 - * asset-prf-sinistros-2026 (sinistros rodoviários, DATEX II)
 - * asset-dnit-pavimentacao-2026 (condições de pavimento, DATEX II)
- Contract Definitions:
 - * Política pública (gratuito, órgãos públicos)
 - * Política comercial (pago, concessionárias)

- Data Addresses:
 - * HTTP endpoints para Apache Camel staging
 - * S3/MinIO buckets

Saída: Docker Compose para dev, Kubernetes manifests para prod

Prompt 2.4: Políticas ODRL — Templates

Gere templates JSON-LD de políticas ODRL para o MVP da RNDT.

- 1. POLÍTICA PÚBLICA (PRF Sinistros):
 - Permissão: use para fins estatísticos e planejamento
 - Proibição: revenda, reidentificação de pessoas (LGPD)
 - Restrição: órgãos públicos e pesquisa acadêmica
 - Vigência: 30 dias
- 2. POLÍTICA COMERCIAL (DNIT Pavimentação):
 - Permissão: use para fins comerciais
 - Obrigação: compensação mensal (R\$ 5.000)
 - Restrição: não redistribuir
 - Vigência: 90 dias, renovável
- 3. POLÍTICA RESTRITA (Dados Sensíveis):
 - Permissão: use apenas com securityClearance Nivel_2
 - Proibição: qualquer derivação
 - Vigência: 7 dias
 - Auditoria obrigatória

Saída: contextos semânticos, exemplos de uso, validação contra schema ODRL

📌 PROJETO CONS-UFPE-003 | Squad 3: Consumer (UFPE)

Objetivo

Preparar o nó consumidor da UFPE para descobrir, negociar e receber dados até o **Dia 15**.

Entregáveis

#	Entregável	Prazo	Critério de Aceite
3.1	EDC Consumidor (UFPE) deployado	D10	Control Plane + Data Plane, mTLS
3.2	Certificado ICP-Brasil configurado	D12	Registro no Keycloak, token válido
3.3	Pipeline de ingestão pós-recebimento	D14	DATEX II → PostgreSQL/PostGIS
3.4	Coleção Postman validada	D15	Todos os endpoints testados

Dependências

Precisa de	Interface	Mitigação até D15
INFRA-CORE-001 (Keycloak)	OAuth2 + mTLS	Usar Omejdn mock local
INFRA-CORE-001 (CKAN)	REST API	Usar nginx + JSON estático
DATA-PROV-002	EDC Provedor (DSP)	Usar mock DSP (nginx + JSON)

Prompts LLM

Prompt 3.1: Configuração Eclipse EDC — Consumidor

Gere a configuração do Eclipse Dataspace Connector como Consumidor da UFPE.

Requisitos:

- application.properties:
 - * Control Plane: DSP client, catalog request
 - * Data Plane: HTTP receiver, mTLS
 - * Identity: Keycloak client (CNPJ UFPE)
 - * Callback URLs para negociação
- Docker Compose:
 - * EDC consumidor
 - * PostgreSQL local para cache de contratos
 - * Nginx reverse proxy com mTLS
- Scripts de automação:
 - * catalog-request.sh: descobre assets no Broker
 - * contract-negotiation.sh: negocia contrato ODRL
 - * transfer-request.sh: solicita transferência de dados
 - * verify-receipt.sh: valida recibo na Clearing House
- Configuração de certificados ICP-Brasil (e-CNPJ UFPE)

Prompt 3.2: Pipeline de Ingestão Pós-Recebimento

Gere um pipeline Apache Camel para processar dados recebidos do EDC consumidor.

ENTRADA: Arquivos DATEX II (JSON/XML) recebidos via Data Plane

PROCESSAMENTO:

- Parse DATEX II → estrutura tabular
- Validação de schema (XSD/JSON Schema)
- Enriquecimento: cruzar com base geográfica (PostGIS)
- Transformação: converter para Parquet/CSV
- Carga: inserir em PostgreSQL + PostGIS

SAÍDA: Tabelas analíticas prontas para BI

REGRAS:

- Batch e streaming
- Log de auditoria (proveniência, timestamp)
- Retenção conforme política ODRL (auto-delete após vigência)
- Notificação de sucesso/falha

Saída: rotas Camel, processors, configuração de datasource, testes

Prompt 3.3: Coleção Postman — Fluxo Consumidor

Gere uma coleção Postman completa para o fluxo do Consumidor na RNDT.

COLLECTION: "RNDT Consumer — UFPE"

FOLDERS:

1. Autenticação:

- POST /realms/RNDT-MVP/protocol/openid-connect/token
- Headers: mTLS client certificate
- Tests: validar JWT, claims, expiração

2. Descoberta (CKAN):

- GET /api/3/action/package_search?q=sinistros
- GET /api/3/action/package_show?id=asset-prf-sinistros-2026
- Tests: validar metadados mobilityDCAT-ap

3. Negociação (EDC DSP):

- POST /api/v1/dsp/catalog/request
- POST /api/v1/dsp/negotiation/request
- POST /api/v1/dsp/contractagreements
- Tests: validar ODRL, assinaturas

4. Transferência:

- POST /api/v1/dsp/transferprocess
- GET /api/v1/public/stream/{transferId}
- Tests: validar hash SHA-256, recibo

5. Auditoria:

- POST /api/v1/clearinghouse/log
- GET /api/v1/clearinghouse/receipt/{id}

ENVIRONMENTS:

- RNDT-Dev (localhost)
- RNDT-Staging (k8s interno)
- RNDT-Prod (produção)

Saída: pre-request scripts, tests, variáveis de ambiente, documentação

🌈 PROJETO GOV-PORTAL-004 | Squad 4: Gov, Portal & Playbook

Objetivo

Criar o Portal da RNDT, dashboards e documentação de onboarding até o **Dia 30**.

Entregáveis

#	Entregável	Prazo	Critério de Aceite
4.1	Tema CKAN (identidade RNDT)	D10	Landing page, busca, dataset page
4.2	Páginas institucionais	D12	“Como Participar”, sobre, FAQ
4.3	Harvester CKAN configurado	D14	Colheita automática do EDC Provedor
4.4	Dashboards Superset	D20	3 dashboards operacionais
4.5	Templates ODRL	D15	5 templates parametrizáveis
4.6	Coleção Postman	D20	Onboarding passo-a-passo
4.7	Playbook de Onboarding	D25	Markdown, diagramas, comandos
4.8	Documentação de governança	D30	Políticas, normas, FAQ jurídico

Dependências

Precisa de	Interface	Mitigação
INFRA-CORE-001 (CKAN)	Tema + plugins	Desenvolver com CKAN local
INFRA-CORE-001 (Superset)	SQL + dashboards	Desenvolver com CSV de exemplo
DATA-PROV-002	Metadados mobilityDCAT-ap	Usar JSON-LD estático

Prompts LLM

Prompt 4.1: Tema CKAN — Portal da RNDT

```
Gere um tema CKAN completo (extensão ckanext-rndt-theme).

ESTRUTURA:
ckanext/rndt_theme/
├─ templates/
│   ├── home/index.html (landing page)
│   ├── home/snippets/rndt_search.html (busca central)
│   ├── home/snippets/rndt_categories.html (Sinistros, Pavimento, Tráfego)
│   ├── package/read.html (botão "Solicitar Acesso via Data Space")
│   └─ page/como-participar.html (tutorial onboarding)
├─ public/
│   ├── css/rndt.css (cores do Ministério dos Transportes)
│   ├── img/ (logo RNDT, ícones)
│   └─ js/rndt.js (interatividade)
└─ plugin.py

LANDING PAGE:
```

- Hero: "Rede Nacional de Dados de Transporte"
- Barra de busca central (estilo dados.gov.br)
- Cards de categorias com ícones
- Seção "Como Participar" com passos ilustrados
- Footer: logos PRF, DNIT, ANTT, Serpro, UFPE

DATASET PAGE:

- Metadados amigáveis (título, descrição, cobertura, formato)
- Botão: "Solicitar Acesso via Data Space" → endpoint DSP
- Aba "Regras de Uso" → resumo ODRL
- Aba "Documentação Técnica" → swagger/openapi

Saída: templates Jinja2, CSS Bootstrap 5, plugin.py, setup.py

Prompt 4.2: Dashboards Superset — Governança

Gere queries SQL e configurações de dashboard para Apache Superset.

DASHBOARD 1: "Saúde do Portal RNDT"

- Métrica: buscas por dia (CKAN logs)
- Gráfico: linha temporal
- Métrica: datasets mais populares (top 10)
- Gráfico: bar chart horizontal
- Métrica: taxa de conversão (busca → acesso)
- Filtro: período, categoria, órgão

DASHBOARD 2: "Saúde do Data Space"

- Métrica: nós ativos (conectores online)
- Gráfico: mapa geográfico
- Métrica: contratos ODRL assinados por dia
- Gráfico: linha temporal
- Métrica: volume transferido (GB)
- Gráfico: área chart
- Métrica: taxa de sucesso/falha

DASHBOARD 3: "Auditoria e Compliance"

- Métrica: logs na Clearing House por dia
- Tabela: últimas transferências (hash, timestamp, status)
- Alerta: transferências com hash inválido
- Filtro: órgão, período, tipo de asset

Saída: queries SQL, configurações de charts, JSON de exportação

Prompt 4.3: Playbook de Onboarding

Gere um Playbook de Onboarding em Markdown para novos participantes da RNDT.

Playbook de Onboarding — RNDT Data Space

1. Pré-requisitos

- Certificado digital ICP-Brasil (e-CNPJ/e-CPF)

- Acesso à rede do órgão/empresa
- Kubernetes cluster (mínimo: 4 vCPU, 8GB RAM)

2. Onboarding em 4 Passos

Passo 1: Identidade (15 min)

- Solicitar cadastro no Keycloak da RNDT
- Enviar certificado público para o Ministério dos Transportes
- Receber confirmação de homologação

Passo 2: Infraestrutura (30 min)

- Clonar repositório de Helm Charts
- Ajustar values.yaml com seu CNPJ e certificados
- Executar: `helm install rndt-connector ./charts/edc-connector`
- Verificar health checks

Passo 3: Configuração (30 min)

- Cadastrar assets no EDC (usar template JSON)
- Definir políticas ODRL (usar template)
- Configurar Apache Camel para ingestão de dados legados
- Testar endpoint DSP localmente

Passo 4: Integração (30 min)

- Solicitar colheita no CKAN (formulário online)
- Testar negociação com consumidor piloto (UFPE)
- Validar registro na Clearing House

3. Templates

- Helm Chart do EDC (anexo)
- Template de Asset JSON
- Template de Política ODRL
- Script de teste Postman

4. Troubleshooting

- Erro 401: Verificar certificado ICP-Brasil
- Erro 403: Verificar rndtStatus no Keycloak
- Erro 500: Verificar logs do EDC
- Timeout: Verificar firewall e NetworkPolicy

5. Suporte

- Canal Slack: #rndt-onboarding
- Email: rndt-suporte@transportes.gov.br
- Horário: seg-sex, 9h-18h

Saída: diagramas, screenshots, comandos copy-paste, checklists

 PROJETO INTEG-GOLIVE-005 | Integração & Go-Live

Objetivo

Integrar os 4 squads, validar o ciclo DSP e realizar o evento de Go-Live no **Dia 30**.

Entregáveis

#	Entregável	Prazo	Critério de Aceite
5.1	Testes de integração	D20	Conectores falam via Keycloak
5.2	Teste E2E completo	D25	Ciclo DSP validado
5.3	Simulação 3º nó	D28	Onboarding em < 4 horas
5.4	Evento de Go-Live	D30	Demo ao vivo para o Ministro

Prompts LLM

Prompt 5.1: Script de Teste E2E

```
Gere um script de teste end-to-end (Python/Bash) para validar o ciclo completo.

CENÁRIO: UFPE consome dados de sinistros da PRF

PASSOS:
1. Autenticar no Keycloak (mTLS)
2. Buscar asset no CKAN (harvester)
3. Negociar contrato (DSP)
4. Transferir dados (Data Plane)
5. Registrar recibo (Clearing House)
6. Validar dashboard (Superset)

ASSERTS:
- Token JWT válido
- Metadados mobilityDCAT-ap corretos
- Contrato ODRL assinado por ambas as partes
- Hash SHA-256 do dado bate com recibo
- Log aparece no Superset em < 5 min

REPORT: HTML com resultado de cada passo, screenshots, métricas de tempo

Saída: script executável, requirements.txt, README
```

Prompt 5.2: Roteiro do Evento de Go-Live

```
Gere um roteiro detalhado para o evento de Go-Live (30 minutos).

[00:00-02:00] Abertura
- Contexto: silos de dados de transporte no Brasil
- Visão: RNDT como ecossistema federado
```

- Parceiros: PRF, DNIT, ANTT, Serpro, UFPE
- [02:00-08:00] Demo 1: O Portal da RNDT
- Acessar rndt.gov.br no telão
 - Buscar "sinistros BR-101"
 - Mostrar metadados e botão "Solicitar Acesso"
 - Mostrar dashboard Superset em tempo real

- [08:00-18:00] Demo 2: O Ciclo DSP
- Mostrar conector PRF (provedor)
 - Mostrar conector UFPE (consumidor)
 - Executar negociação de contrato (automático)
 - Mostrar transferência de dados em tempo real
 - Mostrar recibo na Clearing House

- [18:00-25:00] Demo 3: Onboarding Rápido
- Simular chegada de novo órgão
 - Mostrar Playbook (4 passos)
 - Executar deploy em < 4 horas (time-lapse)

- [25:00-30:00] Fechamento
- KPIs do MVP (contratos, transferências, nós)
 - Próximos passos (escala, novos órgãos)
 - Agradecimentos

MATERIAL DE APOIO: slides, checklist técnico pré-evento, plano de contingência

ANEXOS

Anexo A: Especificação Técnica das Interfaces

A.1 Identidade (Keycloak / DAPS)

Campo	Especificação	Exemplo
URL do issuer	https://identity.rndt.gov.br/realms/RNDT-MVP	—
Endpoint de token	.../protocol/openid-connect/token	—
Endpoint de certs	.../protocol/openid-connect/certs	—
Algoritmo	RS256	—
Claims obrigatórias	sub, orgao, rndtStatus, securityProfile, iat, exp	—
Formato do sub	urn:govbr:cnpj:{14 dígitos}	urn:govbr:cnpj:00394502000144
rndtStatus	Homologado, Pendente, Bloqueado	—
securityProfile	Base, BaseFree, Trust, TrustPlus	—
Duração do token	300 segundos	—

Mock (Dia 1–14):

```
# docker-compose.yml — Omejdn DAPS
services:
```

```
omejdn:
  image: ghcr.io/omejdn/omejdn-server:latest
  volumes:
    - ./omejdn:/opt/omejdn/config
  ports:
    - "4567:4567"
```

A.2 Catálogo (DSP / CKAN)

Campo	Especificação	Exemplo
Endpoint DSP	<code>https://edc.{orgao}.rndt.gov.br/api/v1/dsp</code>	<code>https://edc.prf.gov.br/api/v1/dsp</code>
Catálogo	<code>.../catalog</code>	—
Método	GET	—
Autenticação	Authorization: Bearer {JWT}	—
Content-Type	application/json	—
Formato	JSON-LD com @context DSpace v0.8	—

Schema da resposta:

```
{
  "@context": "https://w3id.org/dspace/v0.8/context.json",
  "@id": "https://edc.prf.gov.br/catalog",
  "@type": "dcat:Catalog",
  "dct:title": "string",
  "dct:description": "string",
  "dcat:dataset": [{
    "@id": "urn:rndt:asset:{id}",
    "@type": "dcat:Dataset",
    "dct:title": "string",
    "dct:description": "string",
    "dcat:distribution": {
      "@type": "dcat:Distribution",
      "dct:format": "string",
      "dcat:accessService": {
        "@type": "dcat:DataService",
        "dcat:endpointUrl": "string",
        "edc:assetId": "string"
      }
    }
  }]
}
```

Mock (Dia 1–14):

```
{
  "@context": "https://w3id.org/dspace/v0.8/context.json",
  "@id": "https://edc.prf.gov.br/catalog",
  "dcat:dataset": [{
```

```

"@id": "urn:rndt:asset:prf-sinistros-2026",
"dct:title": "Sinistros PRF 2026",
"dcat:accessService": {
  "dcat:endpointUrl": "https://edc.prf.gov.br/api/v1/dsp",
  "edc:assetId": "asset-prf-sinistros-2026"
}
}]
}

```

A.3 Negociação (DSP Contract)

Campo	Especificação	Exemplo
Endpoint	.../negotiation/request	—
Método	POST	—
Request	CatalogRequestMessage (JSON-LD)	—
Response	ContractOffer (JSON-LD) com ODRL	—
Callback	https://edc. {orgao}.rndt.gov.br/api/v1/ dsp/callback	—

Schema do ContractOffer:

```

{
  "@context": "https://w3id.org/dspace/v0.8/context.json",
  "@id": "urn:rndt:offer:{uuid}",
  "@type": "odrl:Offer",
  "odrl:target": "urn:rndt:asset:{id}",
  "odrl:assigner": "urn:govbr:cnpj:{14 dígitos}",
  "odrl:permission": [...],
  "odrl:prohibition": [...],
  "odrl:obligation": [...],
}

```

A.4 Transferência (DSP Data Plane)

Campo	Especificação	Exemplo
Endpoint	.../transferprocess	—
Método	POST	—
Request	TransferRequestMessage (JSON-LD)	—
Protocolo	HttpProxy (mTLS)	—
Payload	DATEX II (JSON/XML) ou GTFS	—

A.5 Auditoria (Clearing House)

Campo	Especificação	Exemplo
Endpoint	https://clearing.rndt.gov.br/api/v1/log	—
Método	POST	—
Body	LogMessage (JSON-LD)	—
Campos obrigatórios	issuerConnector, issued, correlationMessage, payloadDescription, proof	—
Hash	SHA-256 do payload	—
Assinatura	JWS com certificado ICP-Brasil	—

Mock (Dia 1–14):

```
POST http://localhost:8081/api/v1/log
Response: { "status": "REGISTERED", "timestampProof": "2026-06-20T10:35:05Z" }
```

A.6 Infraestrutura (Helm Charts)

Campo	Especificação	Exemplo
Namespace	rndt-{squad}-{ambiente}	rndt-infra-prod
Labels	app.kubernetes.io/name, component, part-of: rndt-mvp	—
Service names	{component}-rndt	keycloak-rndt
Portas	Keycloak: 8080, CKAN: 5000, EDC: 8181, Clearing: 8080	—
ConfigMap	{component}-config	keycloak-config
Secret	{component}-secrets	keycloak-secrets

values-base.yaml (template comum):

```
global:
  rndt:
    environment: prod
    domain: rndt.gov.br
    imageRegistry: registry.rndt.gov.br

ingress:
  enabled: true
  className: nginx
  tls:
    enabled: true
    certManager: true

resources:
  requests:
    cpu: 100m
    memory: 256Mi
```

```
limits:
  cpu: 1000m
  memory: 1Gi

probes:
  liveness:
    path: /health
    initialDelaySeconds: 30
    periodSeconds: 10
  readiness:
    path: /ready
    initialDelaySeconds: 5
    periodSeconds: 5
```

Anexo B: Mock Server

B.1 Estrutura

```
mock-server-rndt/
├─ docker-compose.yml
├─ nginx/
│   ├─ nginx.conf
│   └─ static/
│       ├─ keycloak/certs.json
│       ├─ dsp/catalog.json
│       ├─ dsp/offer.json
│       ├─ dsp/agreement.json
│       ├─ clearinghouse/receipt.json
│       └─ ckan/package_search.json
└─ omejdn/
    ├─ clients.yml
    └─ keys/
```

B.2 Docker Compose

```
version: '3.8'

services:
  omejdn:
    image: ghcr.io/omejdn/omejdn-server:latest
    volumes:
      - ./omejdn:/opt/omejdn/config
    ports:
      - "4567:4567"
    networks:
      - rndt-mock

  dsp-mock:
    image: nginx:alpine
    volumes:
```

```

- ./nginx/static:/usr/share/nginx/html
- ./nginx/nginx.conf:/etc/nginx/nginx.conf
ports:
  - "8080:80"
networks:
  - rndt-mock

clearing-mock:
  image: nginx:alpine
  volumes:
    - ./nginx/static/clearinghouse:/usr/share/nginx/html
  ports:
    - "8081:80"
  networks:
    - rndt-mock

networks:
  rndt-mock:
    driver: bridge

```

B.3 Postman Collection de Referência

```

{
  "info": {
    "name": "RNDT — Interface Validation (Mock)",
    "description": "Valida as interfaces do IDD contra mocks",
    "version": "1.2"
  },
  "item": [
    {
      "name": "01 — Identidade",
      "item": [{
        "name": "Obter token JWT (Omejdn)",
        "request": {
          "method": "POST",
          "header": [{"key": "Content-Type", "value": "application/x-www-form-urlencoded"}],
          "url": "http://localhost:4567/token",
          "body": {
            "mode": "urlencoded",
            "urlencoded": [
              {"key": "grant_type", "value": "client_credentials"},
              {"key": "client_id", "value": "urn:govbr:cnj:00394502000144"},
              {"key": "scope", "value": "openid"}
            ]
          }
        },
        "event": [{
          "listen": "test",
          "script": {
            "exec": [
              "pm.test('Status 200', () => pm.response.to.have.status(200));",

```

```

        "pm.test('JWT válido', () => {",
        "  const json = pm.response.json();",
        "  pm.expect(json.access_token).to.exist;",
        "  pm.expect(json.token_type).to.eql('Bearer');",
        "});",
        "pm.test('Claims obrigatórias', () => {",
        "  const jwt = pm.response.json().access_token;",
        "  const payload = JSON.parse(atob(jwt.split('.')[1]));",
        "  pm.expect(payload.sub).to.exist;",
        "  pm.expect(payload.rndtStatus).to.exist;",
        "  pm.expect(payload.securityProfile).to.exist;",
        "});",
      ],
    },
  ],
},
{
  "name": "02 — Catálogo",
  "item": [{
    "name": "Descobrir assets (DSP)",
    "request": {
      "method": "GET",
      "header": [{"key": "Authorization", "value": "Bearer {{token}}"}],
      "url": "http://localhost:8080/dsp/catalog"
    },
    "event": [{
      "listen": "test",
      "script": {
        "exec": [
          "pm.test('Status 200', () => pm.response.to.have.status(200));",
          "pm.test('JSON-LD válido', () => {",
          "  const json = pm.response.json();",
          "  pm.expect(json['@context']).to.exist;",
          "  pm.expect(json['dcat:dataset']).to.be.an('array');",
          "});",
        ]
      }
    ]
  }
  ]
},
],
"variable": [
  {"key": "token", "value": ""}
]
}

```

Componente	Versão	Interface	Gap	Mitigação
Keycloak	24.x	OAuth2/OIDC, mTLS, JWT	✅ Nativo	—
Omejdn DAPS	1.x	DAPS nativo para EDC	✅ Nativo	Usar enquanto Keycloak não está pronto
CKAN	2.10	REST API, ckanext-dcat	⚠️ Com plugin	Instalar ckanext-dcat + ckanext-harvest
Eclipse EDC	0.7.x	DSP v0.8	✅ Nativo	—
IDS Clearing House	Fraunhofer ISST	DSP, JSON-LD, JWS	✅ Nativo	—
Apache Superset	3.x	SQL, REST API	✅ Nativo	—
Apache Camel	4.x	CSV, HTTP, S3	✅ Nativo	—

Anexo D: Repositórios de Referência

- Eclipse EDC: <https://github.com/eclipse-edc/Connector>
- CKAN: <https://github.com/ckan/ckan>
- ckanext-dcat: <https://github.com/ckan/ckanext-dcat>
- ckanext-harvest: <https://github.com/ckan/ckanext-harvest>
- Keycloak: <https://github.com/keycloak/keycloak>
- Omejdn DAPS: <https://github.com/omejdn/omejdn-server>
- Apache Camel: <https://github.com/apache/camel>
- Apache Superset: <https://github.com/apache/superset>
- IDS Clearing House: <https://github.com/International-Data-Spaces-Association/IDS-clearing-house-service>

Anexo E: Padrões e Especificações

- IDS RAM 4.0: <https://internationaldataspaces.org>
- Dataspace Protocol: <https://w3id.org/dspace/v0.8>
- DATEX II: <https://datex2.eu>
- mobilityDCAT-ap: <https://w3id.org/transport/mobilityDCAT-ap>
- ODRL: <https://www.w3.org/ns/odrl/2/>
- DCAT-AP: <https://joinup.ec.europa.eu/collection/semantic-interoperability-community-semic/solution/dcat-application-profile-data-portals-europe>
- JSON-LD: <https://www.w3.org/TR/json-ld/>
- JWT: <https://tools.ietf.org/html/rfc7519>
- JWS: <https://tools.ietf.org/html/rfc7515>

Anexo F: Documentos Base

- Documento de Escopo v1.6 (anexo original)
- Documento Técnico de Referência — Arquitetura, Componentes e Guia de Implementação (anexo original)

*Elaborado para o Ministério dos Transportes / RNDT — Data Space de Transporte do Brasil Versão 2.0
— com Interface Definition Document (IDD) completo Junho de 2026*